Cyshield Meeting Brief

Topic 6.4: Zero-Downtime Fleet-Wide Crypto-Agility Orchestrator

Narrowed scope, literature and novelty, methodology, and results

Team Developers • Supervisor: Dr. Moustafa Youssef • October 26, 2026

At a glance

One question, two halves: *when a post-quantum (PQ) migration runs into failures, what should a client do?*

1. **Firewall-behavior prediction model (as agreed).** Predict how a firewall or middlebox will treat a PQ handshake on a given path, before the client commits to a migration tier. The method is fully designed; data collection is next.
2. **Fallback safety (new evidence). 653 of 4,573 PQ-capable sites in the top 10,000 (14.3%, 95% CI 13.3% to 15.3%)**, and 20.4% of the top 1,000, give a classical connection to a client that falls back the way OpenSSL's own manual recommends. **88.8% sit behind Akamai and Fastly.**

Contents

1 Summary 1

2 Background: why this problem exists 2

3 Part 1: the firewall-behavior prediction model 4

4 Part 2: is falling back safe? 10

5 How the two parts connect 12

6 Related work and novelty 14

7 Plan, timeline and deliverables 15

8 Ethics, disclosure and constraints 15

9 Decisions and support we need from Cyshield 15

1 Summary

This brief covers the narrowed scope requested: the topic, the literature and novelty, the methodology, and the results already in hand.

What we bring.

1. **Firewall-behavior prediction model (as agreed).** Predict how a firewall or middlebox will treat a PQ handshake on a given path, before the client commits to a migration tier (classical, hybrid or PQ-only). The method is fully designed: problem statement, outcome labels, features, statistics and tooling. Data collection is the next step.
2. **Fallback safety (new evidence).** When a client falls back after a failure, does it keep PQ? We measured it: 653 of 4,573 PQ-capable sites in the top 10,000 (14.3%), and 20.4% of the top 1,000, give a classical connection to a client that falls back the way OpenSSL's manual recommends; 88.8% of them sit behind Akamai and Fastly. An unmodified OpenSSL 3.5.6 client confirmed it: classical in 18 of 18 handshakes to those CDNs, PQ in 9 of 9 to Google, Cloudflare and Microsoft.

How they fit. To fall back safely, a client needs two facts about a destination: *will PQ fail on this path?* (the prediction model) and *if I fall back, will this server still pick PQ?* (the fallback measurement). Together they make a working program: a tool that profiles paths and servers and decides, per destination, whether falling back is safe.

Our plan. The prediction model goes first so it finishes quickly; the fallback policy proceeds in parallel. We merge them into one system if time allows; otherwise they become two papers.

What we need decided today:

1. Merge the two now, or run them in parallel and merge later.
2. Access to commercial firewalls or TLS inspection appliances, and a route to publish results about them.
3. Agreement on the modeling path if the available data limits what the model can generalize to.
4. Help with coordinated disclosure, especially to Link.NET, and a second vantage point for scanning.

2 Background: why this problem exists

2.1 The threat: harvest now, decrypt later

A quantum computer able to break today's public-key cryptography (RSA, Diffie-Hellman, elliptic curves) does not exist yet, but an adversary does not need one today: encrypted traffic can be recorded now and decrypted later. Mosca [5] states the deadline as

$$X + Y > Z,$$

where X is how long data must stay confidential, Y how long migration takes, and Z how many years until a cryptographically relevant quantum computer. If $X + Y > Z$, data is exposed before migration finishes, even if it starts today. Gupta and Mittal [6] quantify the cost of waiting: starting migration in 2030 instead of 2026 raises the expected exposed fraction of data from 0.37 to 0.54 (figure to confirm against the paper before submission).

2.2 The standards timeline

- **2016, NIST IR 8105** (Chen, Jordan, Liu, Moody, Peralta, Perlner, Smith-Tone) [1]: put crypto-agility on the agenda, the ability to swap algorithms quickly before any replacement was final.
- **2016 to 2024, the NIST PQC competition**, ending in:

- **FIPS 203 (ML-KEM, 2024)** [2]: the key-encapsulation standard, derived from CRYSTALS-Kyber, with security based on the Module-LWE lattice problem.
- **FIPS 204 (ML-DSA)** [3]: a signature standard based on Dilithium.
- **FIPS 205 (SLH-DSA)** [4]: a hash-based signature standard (SPHINCS+), a conservative backup in case lattice schemes are broken.

2.3 Why not switch overnight

1. **Size.** A hybrid X25519MLKEM768 key share is about 1.2 KB (1,216 bytes) against 32 bytes for classical X25519, enough to push the TLS ClientHello past the path's Maximum Transmission Unit and fragment it into several packets. Many firewalls and load balancers were never tested against fragmented handshakes of this shape, and silently break.
2. **Unknown unknowns.** Nobody has decades of production experience with these algorithms, so the industry goes through a **hybrid** stage that combines the old and new algorithms, so that breaking either alone is not enough [8]. The migration path is **Classical, then Hybrid, then Post-Quantum-only**, not one jump.

This is the pattern the *floor* concept depends on: a floor of Hybrid means the system is never allowed to fall back to pure Classical once it has reached Hybrid.

2.4 Why this is a crypto-agility problem

NIST chose the algorithms; the hard part is operational: how a real organization, with a heterogeneous fleet and client-side middleboxes it does not control, rolls out a cryptographic change at scale, safely and reversibly, without taking production down. That operational question is what crypto-agility means, and it is the subject of this project.

2.5 Lessons from past failures

Case	What happened	What it cost	What this project does differently
Chrome 124 (April 2024)	After a staged rollout from Chrome 116 (August 2023), making hybrid X25519+Kyber the desktop default still broke connections behind middleboxes the trial had not caught, documented for Fortinet FortiGate and Palo Alto devices [11]; Google pointed administrators to a temporary policy to disable it. tldr.fail [12] exists because this class of bug is common	Broken handshakes, a public incident	Predict failure before rollout, per cohort, not after
Cloudflare rollout	A deliberately staged, multi-year rollout (PQ offered from October 2022; over half of human traffic PQ by October 2025). Radar publishes the failure taxonomy splitClientHello, hrrFailure, unknownKeyshare [15]. Expansion decisions relied on engineers watching dashboards until Automatic Key Exchange (September 2026) automated per-origin rollback on failure rates [14]	Years of staff time, hard for smaller teams to reproduce	Replace judgment with a quantified, automatable predictive signal
Qadri (2026) [9]	A layered multi-cloud architecture: crypto abstraction layer, policy-driven orchestration engine, centralized HSM control plane, Cryptographic Bill of Materials (CBOM) monitoring module, migration state machine	By its own admission never empirically validated: a hypothesis, not a result	Build and empirically test the predictive component first
Gupta and Mittal (2026) [6]	The most complete process model: Discovery, Design, Pilot, Rollout, Deprecate, with a canary pattern from Kayenta and Spinnaker [16]. As our survey notes, "there is little agreement on how agility should be measured, or on how an organization can tell whether it is ready to switch algorithms quickly"	No way to compare readiness; assumes device behavior is already known	Define and validate a concrete, measurable prediction target

3 Part 1: the firewall-behavior prediction model

3.1 Where the project started and why it narrowed

The original idea was a *zero-downtime fleet-wide crypto-agility orchestrator*: a system that watches an organization's fleet migrate from classical to PQ algorithms, decides when it is safe to expand, detects when something goes wrong, and guarantees (not just hopes) that it never rolls back below a human-set safety floor. Building the decision layer, the monitor, the shield and a realistic rollout engine is several projects' worth of work, so the scope narrowed to the piece everything else depends on and that does not yet exist in the literature: **predicting how a firewall or middlebox will behave when it meets a given PQ migration tier, before committing to that tier.** The three-layer system becomes a later consumer of this prediction.

3.2 The original three-layer architecture (the eventual consumer)

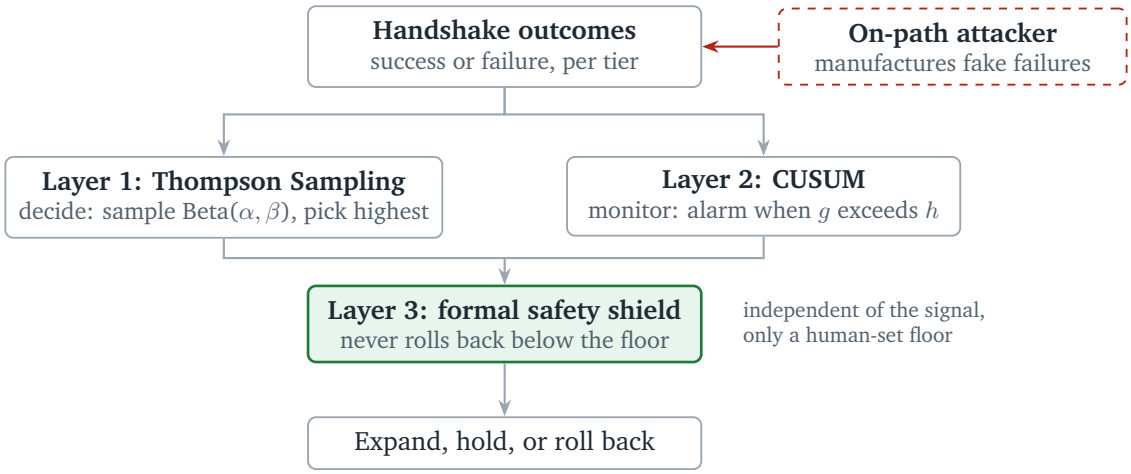


Figure 1: Decide, monitor, enforce: the three layers the prediction will feed. The attacker corrupts the one signal both upper layers read; only the shield is independent of it.

- **Layer 1, Thompson Sampling (decision):** for each tier, keep a Beta(α, β) belief; sample a draw from each, pick the highest, try it, observe success or failure, and add one to α or β.
- **Layer 2, CUSUM (monitor):** runs alongside Layer 1 and watches for a sudden shift that slow-moving α and β counts would notice late.
- **Layer 3, formal safety shield (enforcement):** a small, provably correct component downstream of both: a rollback is blocked if it would go below the human-set policy floor, for every possible input, not only for tested cases.
- **The attacker:** an on-path adversary who manufactures handshake failures to corrupt what the upper layers observe and force an unwarranted, permanent rollback. The shield is the structural defense against the worst case; corruption-robust bandits make Layer 1 harder to fool.

3.3 The theory behind it

Multi-armed bandits. A row of slot machines with unknown payout rates: every pull spent testing a bad machine is a pull not spent on a good one (the exploration and exploitation trade-off). Here each arm is a migration tier or cohort and a payout is a successful handshake. Lineage: Thompson [17] proposed sampling from the current belief, originally for clinical trials; Auer, Cesa-Bianchi and Fischer [19] formalized UCB1, an optimistic non-Bayesian alternative; Russo et al. [18] is the standard tutorial on Thompson Sampling; Agrawal and Goyal [21] gave the first finite-time regret bounds; Auer, Cesa-Bianchi, Freund and Schapire [20] introduced EXP3 for adversarial payouts.

Beta-Bernoulli mechanics.

$$\alpha = 1 + \#\text{successes}, \qquad \beta = 1 + \#\text{failures}.$$

Beta(1, 1) is the uniform distribution, the honest statement of total ignorance; Beta(0, 0) is not a valid distribution; the Jeffreys prior Beta(0.5, 0.5) was considered and rejected because it assumes outcomes cluster near 0% or 100%. As data accumulates the distribution narrows around the true rate.

Why draw rather than pick the maximum. A greedy rule stops exploring once one arm looks good by chance, and it is trivially exploitable: an attacker who injects a few failures learns the threshold once. Random draws keep exploring and make the next choice unpredictable. The

loop is: sample from each arm, pick the highest sample, try it, observe, update. A standard cryptographically secure pseudorandom generator is sufficient; schemes based on prime sequences would make the system more predictable, not less.

Corruption-robust bandits. Lykouris, Mirrokni and Paes Leme [22] model an adversary corrupting up to C observations; Gupta, Koren and Talwar [23] show regret degrading only additively in C; Zimmert and Seldin [24] give a best-of-both-worlds algorithm (Tsallis-INF) that also covers corruption; Xu et al. [25] study robust Thompson Sampling against reward poisoning, with a related corruption-robust Thompson Sampling paper at AISTATS 2026; Bogunovic et al. [26] extend robustness to linear bandits.

CUSUM change-point detection. Thompson Sampling answers "which arm next"; CUSUM answers "has something changed", such as a firmware update, an attacker starting, or a path change. Page [27] introduced it for industrial quality control; Lorden proved it asymptotically optimal and Moustakides exactly optimal [28]; Gong, Lee, Cheng and Xie [29] extend it with neural components (preprint). The score is

$$g_t = \max\left(0,\ g_{t-1} + x_t - \nu\right), \qquad \text{alarm if } g_t > h,$$

where clipping at zero stops a run of good observations from hiding a later bad streak. The threshold h trades the Expected Detection Delay (EDD) against the Average Run Length to a false alarm (ARL_0), much like precision against recall. The two pair naturally: Thompson Sampling gives a stable long-run estimate, CUSUM a fast tripwire.

Formal safety shields. Testing shows correct behavior on the cases tried; a shield is a separate, deliberately simple component mathematically proven to enforce one hard constraint whatever the learned layer does. Alshiekh et al. [30] originated shielding for reinforcement learning; Jansen et al. [31] extended it to probabilistic shields that reason under uncertainty. The shield is a single comparison (is the proposed tier at or above the floor?) that can be proven once, and it sits downstream of the decision layer, blocking any rollback below the floor whether the alarm was genuine or manufactured.

Scope note. The rollout and rollback machinery, including the shield, is deferred: this phase delivers the predictive signal that machinery will consume.

3.4 The narrowed problem

In one sentence: build a model that predicts, for a given firewall or middlebox and a given migration tier, what will happen to the handshake, using only features observable *before* committing to that tier, and that generalizes to middleboxes the model has never directly seen.

Formal statement.

$$p(y \mid x, t)$$

- y is the outcome class: success, splitClientHello, hrrFailure, unknownKeyshare, degraded-success. The failure classes come from Cloudflare Radar's documented taxonomy [15], so the project models a taxonomy already used in production rather than inventing one. Degraded-success (the handshake works, but on classical crypto) is the class Part 2 measures in the wild.
- x is the feature vector describing the client and middlebox.
- t is the migration tier attempted: classical, hybrid or PQ-only.

Feature taxonomy.

- *Direct characteristics* (mechanically determine the outcome): TLS fingerprint (JA3/JA4, the standard passive fingerprint of the TLS client); TCP MSS and path MTU, which decide

whether a large hybrid ClientHello fragments; offered key_share groups and extension ordering; observed HelloRetryRequest behavior on a prior connection.

- *Indirect characteristics* (correlate with the outcome and allow generalizing to unseen devices): network (ASN) and network type (enterprise, residential, cloud); client software family and version; carrier-grade NAT, a proxy for consumer ISP infrastructure; inferred middlebox firmware age from TLS extension support patterns.

The split matters for the method: direct features are what a deterministic rule can already use; indirect features are where a learned model earns its keep, because no fixed rule captures "this network correlates with this firmware generation."

Why this framing. Our literature survey [36] found that "no existing work brings PQC deployment, automated canary rollout, and robust, safe decision-making together," and, more specifically, no published work predicts middlebox PQ-handshake behavior *before* rollout from observable features. Qadri's CBOM module monitors deployed state after the fact; Gupta and Mittal's canary watches a small cohort without predicting beyond it; Cloudflare's taxonomy is a post-hoc catalog, not a predictive model.

Related work	What it offers	What it leaves for prediction
Cloudflare Radar PQ checker (2026) [15]	Per-host detection of the three failure types	Detects after the fact; no model
tldr.fail [12], pqprobe	Direct tests of one endpoint with large or split ClientHellos	Answers today's hybrid case by testing; no prediction for configurations not deployed yet (ML-KEM-1024, larger or three-segment ClientHellos, PQ certificate chains, multi-packet QUIC Initials)
dMAP (CCS 2025) [35]	Remote fingerprinting of DPI devices with 20 to 40 probes	Censorship focus, no PQ tolerance; a methodological relative
Gupta and Mittal; Qadri (2026)	Process models and architectures	Assume device behavior is already known

Scope boundary. Rollback, rollout staging, Thompson Sampling action selection and the CUSUM and shield machinery are future work. This phase delivers the predictive signal; it does not consume it yet.

3.5 The statistics and ML toolkit

In plain language. For a given firewall characteristic (say, a specific TCP MSS value) we want a number like "92% chance of success" plus an honest sense of how sure we are: 92% from 3 trials means something very different from 92% from 3,000. Grouping similar values into ranges is a good instinct, but raises two questions: where do the range boundaries go, and how do we know two ranges really behave the same? The answers: let an interpretable **decision tree** find the boundaries (it can report, for example, that MSS below 1400 behaves differently from above 1400, a sentence a committee can follow), and use an **equivalence test** to show two ranges genuinely behave alike, since failing to prove a difference is not proving sameness. "Plus or minus a standard deviation" is the wrong spread for a probability. **Testing and ML do different jobs:** real handshakes produce the ground-truth (feature, tier, outcome) observations, and statistics and ML extend from the tested cases to the untested ones with honest uncertainty.

Rigorously.

(a) **Uncertainty on a rate.** A normal "plus or minus SD" interval can leave $[0, 1]$. The **Wilson score interval** stays inside it and is accurate for small samples; the **Beta posterior credible**

interval (k successes of n trials gives Beta$(k+1, n-k+1)$) reuses the same Beta machinery as the bandit layer, one piece of theory doing double duty.

(b) **Range boundaries: CART.** Classification and Regression Trees split the data on the feature threshold that best separates outcome classes (Gini impurity or information gain); the leaves are the equivalence-class candidates. Gradient-boosted trees do this with higher accuracy; a single shallow tree keeps it readable.

(c) **Confirming two ranges are equivalent: TOST.** Two One-Sided Tests, from pharmaceutical bioequivalence testing, positively establish equivalence within a tolerance band defined in advance; two candidate ranges are merged only if their success-rate difference is bounded inside it.

(d) **Guaranteed uncertainty bands: conformal prediction.** Conformal prediction [33, 32] wraps any trained model and gives prediction intervals with a finite-sample coverage guarantee under exchangeability: a 90% interval contains the true outcome 90% of the time when test cases resemble the training cases. Applying it to PQ-migration firewall behavior is itself a citable novelty angle.

(e) **Division of labor.** Testing (lab-injected faults on a controlled testbed, plus real-world signals of the Cloudflare Radar and tldr.fail kind) produces the raw (x, t, y) observations. Statistics and ML then estimate $p(y \mid x, t)$ for observed combinations with valid intervals and generalize to combinations never tested, which a lookup table cannot do.

One pipeline, not competing approaches. A gradient-boosted classifier is the accurate version of "find the thresholds"; a shallow tree is its readable cousin for the report; conformal prediction turns raw probabilities into rigorous ranges; TOST justifies calling two ranges the same. Build order: train the full model first, then extract the shallow tree or TOST-merged buckets for the explanation layer, never the reverse, because hand-picked buckets built before a working model encode assumptions rather than discoveries.

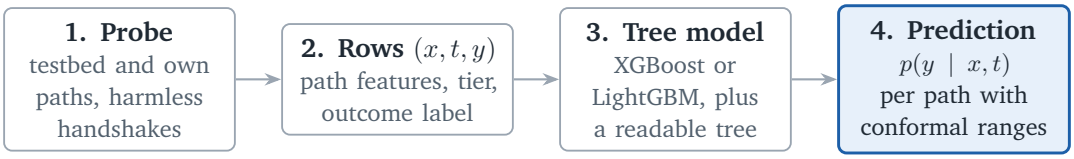


Figure 2: Testbed observations become calibrated, per-path predictions. Commercial firewalls in step 1 need Cyshield's help; open-source middleboxes do not.

3.6 Tooling

The data is tabular (one row per handshake attempt, columns for features, one label column). Grinsztajn, Oyallon and Varoquaux [34] show gradient-boosted trees consistently beat deep networks on exactly this kind of data at project scale (hundreds to low thousands of rows), which is a principled reason for a tree-based primary model.

Tool	Role	Why
Google Colab	Where everything runs	Free, no local setup, shareable notebooks for a five-person team
Python	The language	Standard for everything below
pandas, numpy	Data wrangling	Loading and cleaning (feature, outcome) rows
scikit-learn	Decision trees, Wilson intervals, train and test splits	The shallow, readable tree
XGBoost or LightGBM	The primary predictive model	The accurate classifier for $p(y \mid x, t)$
MAPIE	Conformal prediction wrapper	Valid prediction intervals around the boosted model; no deep learning needed
statsmodels or scipy.stats	TOST equivalence testing	Confirming which buckets are equivalent
PyTorch or TensorFlow	Few-shot or embedding model (secondary)	Only for generalizing to a completely unseen vendor fingerprint, for example a Siamese or prototypical network
TensorFlow Probability	Bayesian uncertainty on a neural model	Approximate credible intervals; conformal prediction gives a provable guarantee on any model, including this one

PyTorch or TensorFlow: either works for the few-shot component; PyTorch has more few-shot example code and papers, which matters more on a semester timeline. Pick one. **What not to do:** do not build the primary predictor as a deep network just because a framework was planned ("the data is tabular and the literature shows trees win here" is the answer to a committee asking why), and do not report raw boosted-tree probabilities as calibrated; the conformal step fixes that.

3.7 Data plan

- **Testbed (ready to run):** a TLS 1.3 origin behind Squid, Suricata, HAProxy, nginx stream, Envoy and Apache Traffic Server, including older versions with documented ClientHello bugs, plus a prober that varies ClientHello size, packet segments and record fragmentation.
- **Public signals:** Cloudflare Radar per-host checks and aggregates; tldr.fail's list of affected products (about ten).
- **Commercial devices:** need Cyshield. The FortiGate-VM free evaluation license disables SSL deep inspection, and Fortinet's and Palo Alto's licenses prohibit publishing benchmark results without consent.
- **Staged modeling:** a tree model on testbed data first; few-shot generalization to unseen vendors as the next stage as data allows; every conformal interval reported with its exchangeability assumption.

3.8 Deliverable and novelty

Deliverable: a working profiler and predictor that probes a path, outputs $p(y \mid x, t)$ with a calibrated range, and explains its decision with a readable rule.

Novelty angle	What it is	Past gap it addresses
Pre-rollout prediction	$p(y \mid x, t)$ from observable features, before committing to a tier	Chrome 124: the fragmentation failure was not predicted before the default rollout
Few-shot generalization	Predicting behavior for a vendor or fingerprint never directly tested	No prior work generalizes beyond its own measured devices
Conformal, valid uncertainty	Guaranteed-coverage prediction intervals, not ad hoc confidence	Staged rollouts relied on human judgment with no quantified confidence

Any one of these is a reasonable senior-project contribution; all three, explicitly connected, read as a coherent system.

4 Part 2: is falling back safe?

This part does not build a rollback system. It measures what happens to PQ protection when clients fall back after failures, which is the situation the prediction model is meant to prevent, and it provides real-world evidence for the degraded-success class.

4.1 The concept

When a PQ handshake fails behind a broken middlebox, clients fall back. Two kinds exist in practice:

- **Full rollback:** remove PQ entirely. Recommended in real user FAQs and issue threads [38].
- **Gentle fallback:** keep PQ on the list of supported groups but stop attaching the large PQ key. OpenSSL 3.5's own manual recommends this setting (`?X25519MLKEM768:DEFAULT`) for networks where large ClientHellos break [38].

Whether the gentle fallback keeps PQ depends on the server:

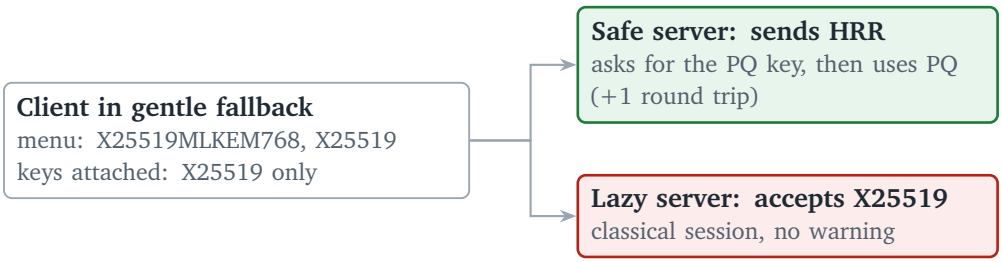


Figure 3: A falling-back client keeps PQ only if the server asks for it. Sources: Go, BoringSSL and OpenSSL source; IETF key share prediction draft.

TLS 1.3 allows both behaviors. The IETF key share prediction draft [37] already describes this mechanism in principle; **nobody had measured how common lazy servers are**, or tied them to fallback.

The attack. TLS 1.3 authenticates the handshake, so an on-path attacker cannot strip PQ from it. It can only make handshakes fail (for example, drop the second packet of the large ClientHello), which looks exactly like a broken firewall. The client's own fallback then does the downgrade. Against a lazy server, a client that always uses the gentle setting is classical even with no attacker at all.

4.2 The experiment

Probe. Phase 1: offer the hybrid key normally; a server that picks it is PQ-capable. Phase 2: list the hybrid first but attach only an X25519 key; HRR means safe, a direct X25519 answer means lazy. **Validated** on three local servers with known behavior (Go and OpenSSL DEFAULT send HRR; an OpenSSL flat list accepts X25519): 3 of 3 correct.

Strict re-check. Every flagged site was retested on the same server IP, three times, phases interleaved, counting only 3-of-3 agreement, so sites with mixed back ends cannot fake a result. **Ethics:** 5 connections per second, one handshake per test, no HTTP requests, one vantage point in Egypt.

4.3 Results

Measure	Pilot, top 1,001 (Oct 18)	Full scan, top 10,000 (Oct 22 to 23)
PQ-capable	386 (38.6%)	4,573 (45.7%)
Flagged lazy, single probe	85 (22.0%)	752 (16.4%)
Confirmed lazy (3 of 3)	**76 (19.7%, CI 16.0 to 23.9%)**	**653 (14.3%, CI 13.3 to 15.3%)**
Networks involved	9	25
Akamai plus Fastly share	88.2%	88.8%

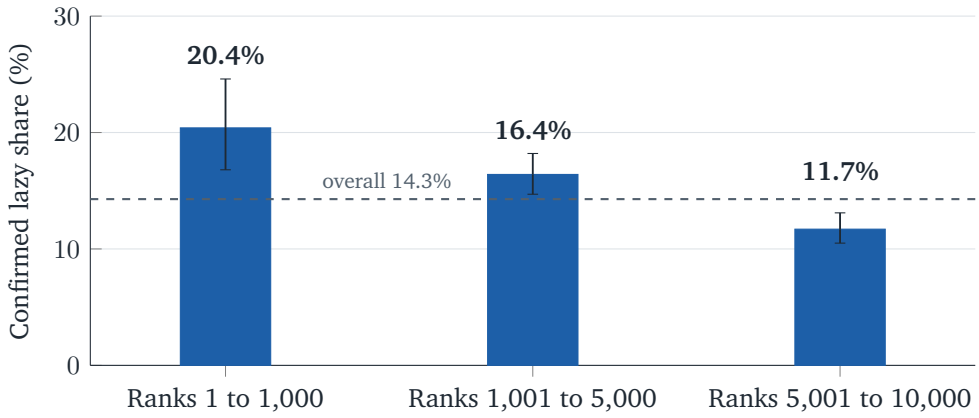


Figure 4: One in five of the top 1,000 PQ sites is lazy, and the rate falls with rank. Confirmed lazy share of PQ-capable sites (82 of 402; 286 of 1,744; 285 of 2,427); whiskers are Wilson 95% intervals. Data: Tranco list 746PX, top 10,000, October 22 to 23, 2026.

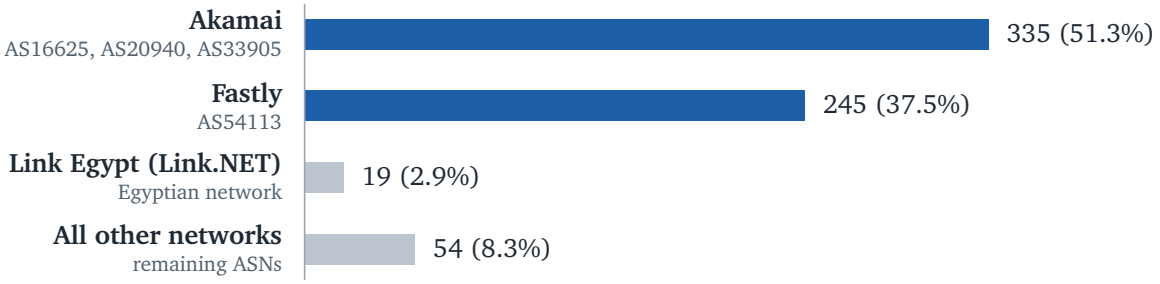


Figure 5: Akamai and Fastly account for 88.8% of the 653 lazy sites, which span 25 networks in total.

The behavior comes from a few shared CDN edge stacks, not hundreds of separate mistakes, which

makes disclosure practical. The 19 Link.NET domains are a local cluster where Cyshield's help would matter.

4.4 Who is exposed: client survey (October 23)

Client	Keys attached	Exposed?
Browsers; Node.js 22, Go 1.24, rustls defaults	Hybrid + X25519	No
OpenSSL manual's workaround	**X25519 only, PQ still listed (374-byte ClientHello)**	**Yes**
Java 21 [41], Python and curl on OpenSSL 3.0	No PQ at all	No PQ to lose

Documented operators using the same pattern: Akamai's origin strategy [39] (August 2025: classical key first, hybrid only after HRR) and Cloudflare's 2023 "supported" origin mode [40]. Exposure is therefore in machine-to-machine traffic (API clients, backends, CDN-to-origin links, device fleets), exactly the setting of a fleet-wide crypto-agility orchestrator.

4.5 Live confirmation with an unmodified client

Sites (OpenSSL 3.5.6, workaround setting, 3 runs each)	Result
Fastly: reddit.com, apache.org, pinterest.com	Classical, 9 of 9
Akamai: ebay.com, intuit.com, tiktok.com	Classical, 9 of 9
Controls: google.com, cloudflare.com, microsoft.com	HRR, then PQ, 9 of 9

Why it stays hidden: an engineer who applies the workaround and tests against Google or Cloudflare sees PQ working, while one in seven PQ sites quietly goes classical.

4.6 Limits we state up front

We measured a server-side precondition, not observed harm; browsers are safe. The per-domain interval is too narrow because sites behind one CDN are not independent, so we report per provider. One vantage point, one point in time (33 sites changed between scans), rank-ordered sample. A random sample will be re-checked with an unmodified client before publication. The attacker-forced fallback is still to be demonstrated, in our own testbed only.

5 How the two parts connect

A client that hits a PQ failure today falls back blindly. A safe decision needs one fact from each part:

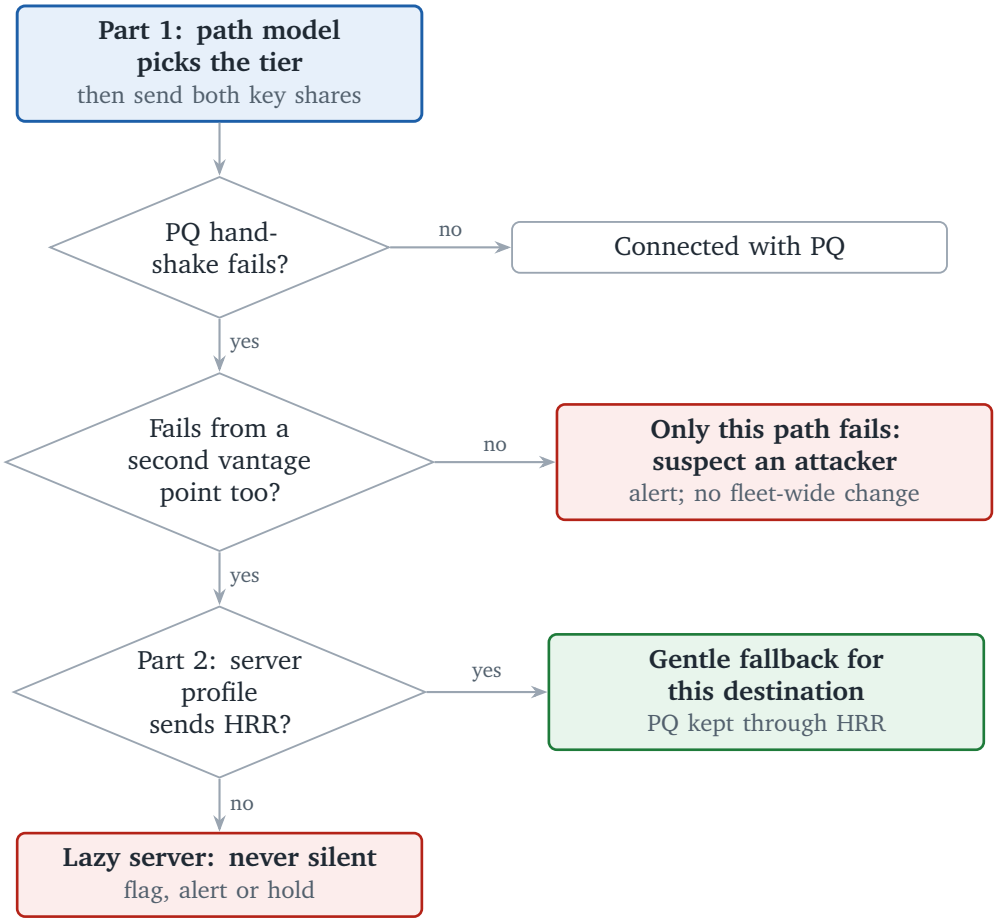


Figure 6: Each part answers one question in the fallback decision.

The merged system (the working program): an operator-controlled client that (1) uses the path model to pick the tier it tries, (2) sends both key shares by default, (3) confirms a failure from a second vantage point before believing it, so an attacker on one path cannot trigger a fallback, (4) uses the server profile to fall back gently only where the server sends HRR, and never silently where it does not, (5) scopes every fallback to one destination, never the fleet, and (6) treats a destination that used to negotiate PQ and now answers classical as an alarm.

This keeps the agreed deliverable (predictive firewall characteristics with a working program) and adds the server half that makes the program safe to act on.

Option	What it means	Trade-off
A. Merge now	One project and one system from the start; two papers (measurement, then system)	Strongest final result; larger scope to manage
B. Parallel, merge later (our plan)	Prediction model is the main deliverable and goes first; the fallback track continues alongside and joins at the policy step	Respects the original agreement; merge depends on time
C. Two separate papers	Prediction paper and fallback measurement paper, no joint system	Lowest risk; loses the combined program

Unified research question (if merged): when a PQ TLS migration falls back after failures, how often does it silently lose PQ protection, and how can a client decide, per destination, whether falling back is safe? Sub-questions: RQ1, how common lazy servers are (answered); RQ2, who is exposed (mostly answered); RQ3, whether cheap probes predict path failures (Part 1, open); RQ4, whether the policy prevents silent downgrades and at what cost (open).

6 Related work and novelty

Work	Part	What it covers	What it leaves for us
IETF key share prediction draft [37]	2	Names the lazy-server mechanism and the risk "even under attacker influence"; proposes server rules and DNS hints	No measurement of prevalence; no link to fallback
Cloudflare Automatic Key Exchange (Sept 2026) [14]	Both	Production per-origin probe-and-rollback loop, about 45 billion connections a day	Reacts to failure rates an attacker can create; treats rollback as a latency cost; internal to Cloudflare
Akamai PQC considerations (Aug 2025) [39]	2	Origin strategy: classical key first, hybrid only after HRR	Does not address origins that never send HRR
Cloudflare Radar PQ checker (May 2026) [15]	1	Detects Split ClientHello, HRR Failure, Unknown Keyshare per host	Failure detection only; a downgraded success looks healthy
tldr.fail [12], pqprobe	1	Direct tests with large or split ClientHellos	No prediction; no policy
dMAP (CCS 2025) [35]	1	Remote DPI fingerprinting with 20 to 40 probes	Censorship focus; no PQ tolerance
Gupta and Mittal [6]; Qadri [9] (2026)	1	Crypto-agility process models and architectures	Not empirically validated; assume device behavior is known
"Nothing Breaks" (Han, 2026) [42]	2	LLM agents remove PQ from configs; no check notices	Authoring-time edits, not runtime fallback or network attackers
AIOpsDoom (USENIX Security 2026) [43]	2	Poisoned telemetry makes an operations agent roll back	No TLS or PQ
PQ deployment measurements, 2025 to 2026 [44]	2	Share of sites supporting hybrid PQ	Never test server selection under a classical-only key share
Corruption-robust bandits [22]; Netflix sequential testing [45]	Both	Learning and monitoring under corrupted or noisy feedback	No PQ setting; useful for verifying failures before believing them

Novelty, Part 1 (to our knowledge): the first model that predicts middlebox behavior toward PQ handshakes from observable features before rollout, with calibrated uncertainty, aimed at configurations that cannot be tested in the field yet.

Novelty, Part 2 (to our knowledge): the first measurement of fallback-induced PQ downgrade at scale (14.3% of PQ-capable top-10,000 sites, concentrated in two CDNs), the first account of which real clients and operators trigger it, and a fallback policy that avoids it. The mechanism is known in principle; its prevalence, its link to fallback and the defense are new. We will recheck before submitting, since preprints in this area appear weekly.

Publishability. ACM PQQS and MAgiCS (PQ-specific; MAgiCS is a Eurocrypt-colocated workshop), the IEEE SVCC Post-Quantum Security Workshop as a realistic workshop target, and IEEE S&P and USENIX Security as stretch goals. The 2026 rounds have closed, so the targets are the 2027 editions. PAM or IMC are possible for the measurement with a second vantage point and repeated scans.

7 Plan, timeline and deliverables

	Part 1: prediction model	Part 2: fallback policy
Done	Method, labels, features, statistics, tooling	Probe, pilot, 10,000-site scan, client survey, live test
By Progress Presentation I (before Nov 5)	Testbed running; first labeled rows; baseline rule-based predictor	Random-sample cross-check with an unmodified client; disclosure started; policy design
By Progress Presentation II (Dec 8) and the specs (Dec 10)	Tree model with calibrated intervals on testbed data; Requirements and Design specifications	Second vantage point; policy engine designed against the testbed; measurement paper drafted
Senior Project II (Spring 2027)	Prediction for undeployed configurations; few-shot generalization as data allows	Policy built, attacked in the testbed and compared with baselines; merge with Part 1

Evaluation of the merged system. Testbed: an OpenSSL 3.5 origin behind open-source middleboxes, plus a scripted attacker that drops ClientHello fragments. Baselines: always send both shares; fall back on any failure (common practice); OpenSSL's gentle workaround; a Cloudflare-style canary loop. Metrics: silent downgrades (target zero), failed connections when the path is genuinely broken, time to recover PQ, false fallbacks caused by the attacker, probes per destination, predictor accuracy against testbed ground truth.

Deliverables. The profiler, predictor and fallback policy as a working program; the testbed and probe suite; the datasets (scan results, ClientHello captures, testbed observations); the Requirements and Design specifications; one or two papers.

8 Ethics, disclosure and constraints

- **Scanning was polite:** 5 connections per second, one TLS handshake per test, no HTTP requests, a public frozen site list (Tranco 746PX).
- **Coordinated disclosure before anything public.** We will privately notify Akamai, Fastly and Link.NET (and OpenSSL about its documentation) with a suggested fix: when a client lists PQ but attaches only a classical key, send HRR instead of accepting classical. We frame it as a prevalence finding of a mechanism already described in the IETF draft, not a new vulnerability class, and allow the usual 90 days. We would welcome Cyshield leading or joining the Link.NET disclosure.
- **The full domain list stays private** until disclosure is complete; this brief names only the nine sites used in the live test.
- **Attacks run only in our own testbed**, against our own servers.
- **Vendor devices:** we will not benchmark commercial firewalls without a license that allows it; FortiGate's evaluation license disables SSL inspection, and Fortinet's and Palo Alto's licenses restrict publishing results.

9 Decisions and support we need from Cyshield

1. **Scope:** merge now (option A), parallel with a later merge (option B, our plan), or two separate papers (option C)?

2. **Fit:** does a profiling tool that predicts path behavior and feeds a safe fallback policy satisfy "predictive firewall characteristics with a working program"?
3. **Devices:** can you give us access to commercial firewalls or TLS inspection appliances, and is there a route to publish (vendor consent or anonymized vendors)?
4. **Model:** is a rule-based or tree model acceptable if the available data cannot support a model that generalizes to unseen vendors?
5. **Disclosure:** would you lead or join coordinated disclosure, especially to Link.NET?
6. **Vantage point:** can we run a second scan from a Cyshield network?
7. **Logistics:** who is our named Cyshield mentor, and how often should we meet before Progress Presentation I?

References

[1] L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, D. Smith-Tone, "Report on Post-Quantum Cryptography," NIST IR 8105, 2016.

[2] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, 2024. https://csrc.nist.gov/pubs/fips/203/final

[3] NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, 2024.

[4] NIST, "Stateless Hash-Based Digital Signature Standard," FIPS 205, 2024.

[5] M. Mosca, "Cybersecurity in an Era with Quantum Computers: Will We Be Ready?", *IEEE Security & Privacy*, 2018.

[6] A. Gupta and A. Mittal, "Operationalizing Post-Quantum Cryptography: A Crypto-Agility Framework," *Cybersecurity*, 2026, doi:10.1186/s42400-026-00579-2.

[7] J. W. Bos, C. Costello, M. Naehrig, D. Stebila, "Post-quantum key exchange for the TLS protocol from the ring learning with errors problem," IEEE S&P, 2015.

[8] D. Stebila, S. Fluhrer, S. Gueron, "Hybrid Key Exchange in TLS 1.3," RFC 9954, IETF, 2026. https://www.rfc-editor.org/rfc/rfc9954.html

[9] A. Qadri, "A Layered Architecture for Multi-Cloud Crypto-Agility Orchestration," working paper, 2026.

[10] P. Ben, "Threat-Aware Machine Learning for Crypto-Agility Anomaly Detection," 2025/2026.

[11] BleepingComputer, "Chrome 124 Breaks TLS Connections on Some Firewalls, VPNs," April 2024; Chromium Blog, "Protecting Chrome Traffic with Hybrid Kyber KEM," 2023, https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html; Chrome 124 middlebox reports, https://blog.boll.ch/?p=8253.

[12] D. Adrian, "tldr.fail: Diagnosing Post-Quantum TLS Failures," Google, 2024. https://tldr.fail

[13] Cloudflare, "Pushing Post-Quantum Cryptography," Cloudflare Blog, 2024 to 2025.

[14] Cloudflare, "Automatic Key Exchange for Origins," 2026. https://blog.cloudflare.com/automatic-key-exchange-for-origins/

[15] Cloudflare Radar, "PQ TLS Bug Detection," May 29, 2026. https://developers.cloudflare.com/changelog/post/2026-05-29-radar-pq-tls-bug-detection/

[16] Spinnaker and Kayenta project documentation, "Automated Canary Analysis."

[17] W. R. Thompson, "On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples," *Biometrika*, 1933.

[18] D. Russo, B. Van Roy, A. Kazerouni, I. Osband, Z. Wen, "A Tutorial on Thompson Sampling," *Foundations and Trends in Machine Learning*, 2018.

[19] P. Auer, N. Cesa-Bianchi, P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, 2002.

[20] P. Auer, N. Cesa-Bianchi, Y. Freund, R. E. Schapire, “The Nonstochastic Multiarmed Bandit Problem,” *SIAM J. Computing*, 2002.

[21] S. Agrawal and N. Goyal, “Analysis of Thompson Sampling for the Multi-armed Bandit Problem,” COLT, 2012; follow-on refinements 2013 and 2017.

[22] T. Lykouris, V. Mirrokni, R. Paes Leme, “Stochastic Bandits Robust to Adversarial Corruptions,” STOC, 2018. https://arxiv.org/abs/1803.09353

[23] A. Gupta, T. Koren, K. Talwar, “Better Algorithms for Stochastic Bandits with Adversarial Corruptions,” COLT, 2019.

[24] J. Zimmert and Y. Seldin, “An Optimal Algorithm for Stochastic and Adversarial Bandits,” AISTATS, 2019.

[25] Z. Xu et al., “Robust Thompson Sampling Algorithms Against Reward Poisoning Attacks,” arXiv:2410.19705, 2024; see also Xu, Wang, Singh, “Corruption Robust Thompson Sampling for Gaussian Bandits,” AISTATS, 2026, https://proceedings.mlr.press/v300/xu26b.html.

[26] I. Bogunovic et al., “Stochastic Linear Bandits Robust to Adversarial Attacks,” AISTATS, 2021.

[27] E. S. Page, “Continuous Inspection Schemes,” *Biometrika*, 1954.

[28] G. Lorden, “Procedures for Reacting to a Change in Distribution,” *Annals of Mathematical Statistics*, 1971; G. V. Moustakides, “Optimal Stopping Times for Detecting Changes in Distributions,” *Annals of Statistics*, 1986.

[29] X. Gong, J. Lee, X. Cheng, Y. Xie, “Neural CUSUM for Online Change-Point Detection,” arXiv:2210.17312, 2022.

[30] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu, “Safe Reinforcement Learning via Shielding,” AAAI, 2018.

[31] N. Jansen, B. Könighofer, S. Junges, A. Serban, R. Bloem, “Safe Reinforcement Learning Using Probabilistic Shields,” CONCUR, 2020. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CONCUR.2020.3

[32] A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021/2023.

[33] V. Vovk, A. Gammerman, G. Shafer, *Algorithmic Learning in a Random World*, Springer, 2005.

[34] L. Grinsztajn, E. Oyallon, G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?” NeurIPS, 2022. https://arxiv.org/abs/2207.08815

[35] Xue et al., “dMAP,” ACM CCS, 2025.

[36] Team Developers, “Literature Survey: Zero-Downtime Fleet-Wide Crypto-Agility Orchestration,” AUC senior project literature survey, 2026.

[37] D. Benjamin et al., “TLS Key Share Prediction,” IETF draft. https://datatracker.ietf.org/doc/draft-ietf-tls-key-share-prediction/

[38] OpenSSL, `SSL_CTX_set1_groups` manual (3.5+), https://manpages.ubuntu.com/manpages/resolute/man3/SSL_CTX_set1_groups.3ssl.html; CVE-2026-2673, https://osv.dev/vulnerability/CVE-2026-2673; OpenSSL discussion #28729, https://github.com/openssl/openssl/discussions/28729.

[39] Akamai, “PQC Implementation Considerations in TLS,” 2025. https://www.akamai.com/blog/security/post-quantum-cryptography-implementation-considerations-tls

[40] Cloudflare, “Post-quantum to origin servers,” 2023. https://blog.cloudflare.com/post-quantum-to-origins/

[41] OpenJDK, JEP 527. https://openjdk.org/jeps/527

[42] Y. Han, “Nothing Breaks: No Single Peer Can Soundly Gate Post-Quantum Delivery,” arXiv:2609.07849, 2026.

[43] D. Pasquini et al., “When AIOps Become ‘AI Oops’,” USENIX Security, 2026. https://arxiv.org/abs/2508.06394

[44] “Mind the Gap: Policy vs Reality in PQ-TLS Deployment,” arXiv:2607.29005, 2026.

[45] M. Lindon, C. Sanden, V. Shirikian, “Rapid Regression Detection in Software Deployments through Sequential Testing,” KDD, 2022. https://arxiv.org/abs/2205.14762